

Тема 10. Особенности архитектуры «клиент-сервер»

Миллионы людей каждый день выходят в сеть Интернет, чтобы почитать новости, пообщаться с друзьями, получить полезную информацию, совершить покупку или оплатить счет. Но большая часть рядовых пользователей даже не догадывается о том, как и с помощью чего они всё это делают, да на самом деле большинству людей это и не нужно, главное, чтобы они получали услугу вовремя и качественно.

Разберемся с концепцией, которая позволяет нам выполнять все эти действия в сети Интернет. Данная концепция получила название «**клиент-сервер**». Как понятно из названия, в данной концепции участвуют две стороны: клиент и сервер. Здесь всё как в жизни: **клиент – это заказчик той или иной услуги, а сервер – поставщик услуг**. Клиент и сервер физически представляют собой программы, например, типичным клиентом является браузер. В качестве сервера можно привести следующие примеры: все HTTP сервера (в частности Apache), MySQL сервер, локальный веб-сервер AMPPS или готовая сборка Denwer (последних два примера – это не просто сервера, а целый набор серверов).

Клиент и сервер взаимодействуют друг с другом в сети Интернет или в любой другой компьютерной сети при помощи различных сетевых протоколов, например, IP протокол, HTTP протокол, FTP и другие. Протоколов на самом деле очень много и каждый протокол позволяет оказывать ту или иную услугу. Например, при помощи HTTP протокола браузер отправляет специальное HTTP сообщение, в котором указано какую информацию и в каком виде он хочет получить от сервера, сервер, получив такое сообщение, отправляет браузеру в ответ похожее по структуре сообщение (или несколько сообщений), в котором содержится нужная информация, обычно это HTML документ.

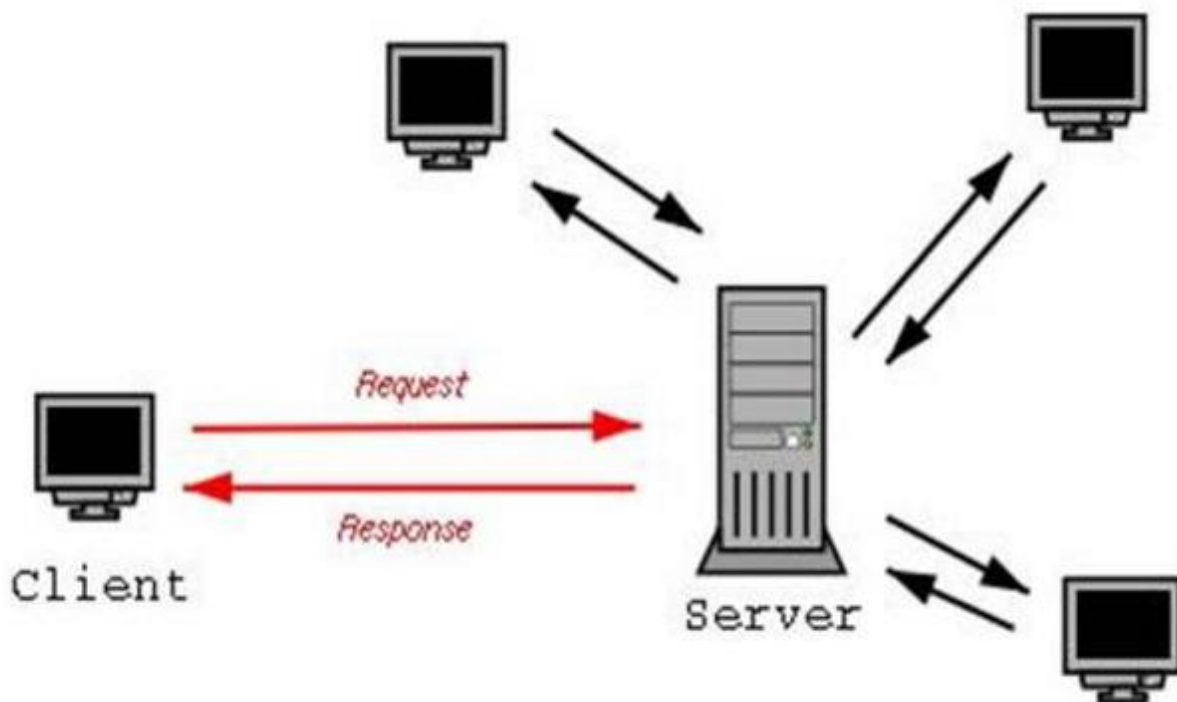
Сообщения, которые посылают клиенты получили названия HTTP запросы. Запросы имеют специальные методы, которые говорят серверу о том, как обрабатывать сообщение. А сообщения, которые посылает сервер получили название HTTP ответы, они содержат помимо полезной информации еще и специальные коды состояния, которые позволяют браузеру узнать то, как сервер понял его запрос.

Мы схематично описали, как взаимодействуют клиент и сервер на седьмом уровне модели OSI, но, на самом деле это взаимодействие происходит на всех семи уровнях. Когда клиент отправляет запрос, сообщение упаковывается, можно представить, что сообщение заворачивается в семь оберток (хотя их может быть намного больше или же меньше), а когда сообщение получает сервер, он начинает эти обертки разворачивать.

Также стоит заметить, что **в основе взаимодействия клиент-сервер лежит принцип того, что такое взаимодействие начинает клиент**, сервер лишь отвечает клиенту и сообщает о том может ли он предоставить услугу клиенту и если может, то на каких условиях. Клиентское программное обеспечение и

серверное программное обеспечение обычно установлено на разных машинах, но также они могут работать и на одном компьютере.

Данная концепция взаимодействия была разработана в первую очередь для того, чтобы разделить нагрузку между участниками процесса обмена информацией, а также для того, чтобы разделить программный код поставщика и заказчика. Ниже вы можете увидеть упрощенную схему взаимодействия клиент-сервер.



Мы видим, что к одному серверу может обращаться сразу несколько клиентов (действительно, на одном сайте может находиться несколько посетителей). Также стоит заметить, что количество клиентов, которые могут одновременно взаимодействовать с сервером зависит от мощности сервера и от того, что хочет получить клиент от сервера.

Многие сетевые протоколы построены на архитектуре клиент-сервер, поэтому в их основе обычно лежат одинаковые или схожие принципы взаимодействия, а разницу мы видим лишь в деталях, которые обусловлены особенностями и спецификой области, для которой разрабатывался тот или иной сетевой протокол.

Основной принцип технологии "клиент-сервер" заключается в разделении функций приложения на три группы:

- ввод и отображение данных (взаимодействие с пользователем);
- прикладные функции, характерные для данной предметной области;
- функции управления ресурсами (файловой системой, БД и т.д.)

Поэтому, в любом приложении выделяются следующие компоненты:

- компонент представления данных

- прикладной компонент
- компонент управления ресурсом

Связь между компонентами осуществляется по определенным правилам, которые называют "протокол взаимодействия".

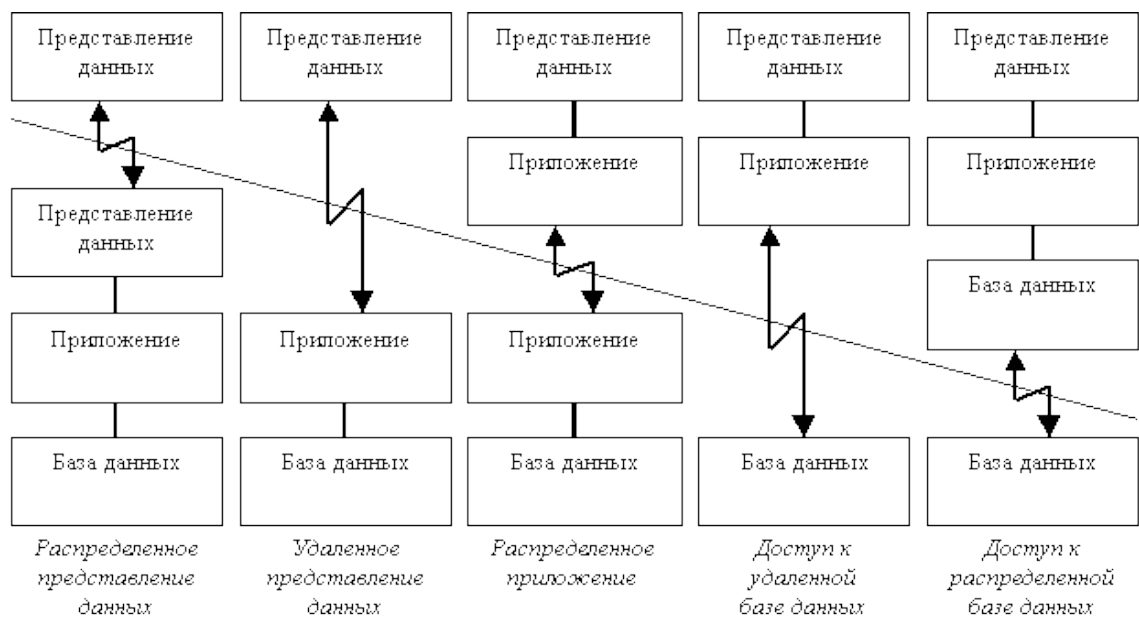
Ответим на вопрос: «зачем веб-мастеру или веб-разработчику понимать концепцию взаимодействия клиент-сервер?». Ответ, естественно, очевиден. Чтобы что-то делать своими руками нужно понимать, как это работает. Чтобы сделать сайт и, чтобы он правильно работал в сети Интернет или хотя бы просто работал, нам нужно понимать, как работает сеть Интернет.

Мы уже упоминали, что большая часть сетевых протоколов имеют архитектуру клиент-сервер. Например, веб-мастеру или веб-разработчику будут интересны протоколы седьмого и шестого уровня эталонной модели. Сетевым администраторам важно понимать, как происходит взаимодействие на уровнях с пятого по второй. Для инженеров связи наибольший интерес представляют протоколы с четвертого по первый уровень модели OSI.

Поэтому если вы действительно хотите быть профессионалом в сфере web, то вначале необходимо понимать, как происходит взаимодействия в сети (именно на седьмом уровне), а уже потом начинать изучать инструменты, которые позволят создавать сайты.

Модели взаимодействия клиент-сервер.

Компанией Gartner Group, специализирующейся в области исследования информационных технологий, предложена следующая классификация двухзвенных моделей взаимодействия клиент-сервер (двухзвенными эти модели называются потому, что три компонента приложения различным образом распределяются между двумя узлами):



Исторически первой появилась модель распределенного представления данных, которая реализовывалась на универсальной ЭВМ с подключенными к ней

неинтеллектуальными терминалами. Управление данными и взаимодействие с пользователем при этом объединялись в одной программе, на терминал передавалась только "картинка", сформированная на центральном компьютере.

Затем, с появлением персональных компьютеров (ПК) и локальных сетей, были реализованы модели доступа к удаленной базе данных. Некоторое время базовой для сетей ПК была архитектура файлового сервера. При этом один из компьютеров является файловым сервером, на клиентах выполняются приложения, в которых совмещены компонент представления и прикладной компонент (СУБД и прикладная программа). Протокол обмена при этом представляет набор низкоуровневых вызовов операций файловой системы. Такая архитектура, реализуемая, как правило, с помощью персональных СУБД, имеет очевидные недостатки - высокий сетевой трафик и отсутствие унифицированного доступа к ресурсам.

С появлением первых специализированных серверов баз данных появилась возможность другой реализации модели доступа к удаленной базе данных. В этом случае ядро СУБД функционирует на сервере, протокол обмена обеспечивается с помощью языка SQL. Такой подход по сравнению с файловым сервером ведет к уменьшению загрузки сети и унификации интерфейса "клиент-сервер". Однако, сетевой трафик остается достаточно высоким, кроме того, по-прежнему невозможно удовлетворительное администрирование приложений, поскольку в одной программе совмещаются различные функции.

Позже была разработана концепция активного сервера, который использовал механизм хранимых процедур. Это позволило часть прикладного компонента перенести на сервер (модель распределенного приложения). Процедуры хранятся в словаре базы данных, разделяются между несколькими клиентами и выполняются на том же компьютере, что и SQL-сервер. Преимущества такого подхода: возможно централизованное администрирование прикладных функций, значительно снижается сетевой трафик (т.к. передаются не SQL-запросы, а вызовы хранимых процедур)..

На практике сейчас обычно используются смешанный подход:

простейшие прикладные функции выполняются хранимыми процедурами на сервере более сложные реализуются на клиенте непосредственно в прикладной программе.

В настоящее время ряд поставщиков коммерческих СУБД объявило о планах реализации механизмов выполнения хранимых процедур с использованием языка Java. Это соответствует концепции "тонкого клиента", функцией которого остается только отображение данных (модель удаленного представления данных).

В последнее время также наблюдается тенденция ко все большему использованию модели распределенного приложения. Характерной чертой таких приложений является логическое разделение приложения на две и более частей, каждая из которых может выполняться на отдельном компьютере. Выделенные части приложения взаимодействуют друг с другом, обмениваясь сообщениями в

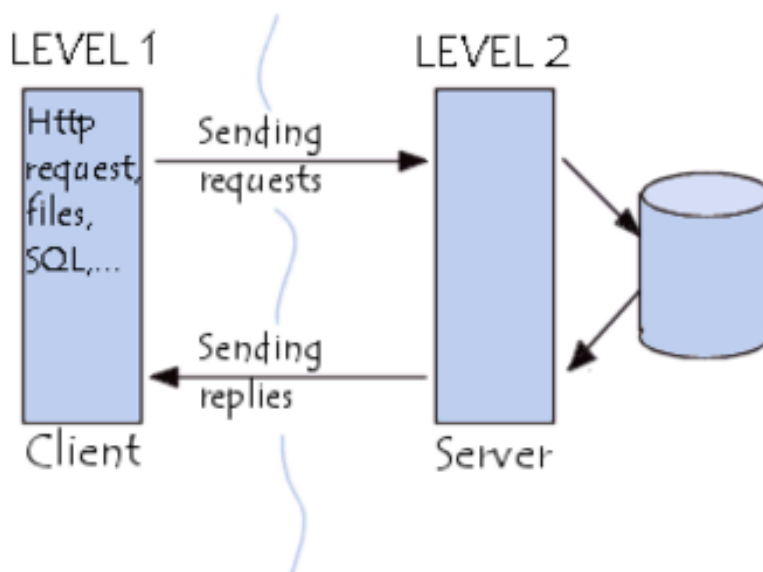
заранее согласованном формате. В этом случае двухзвенная архитектура клиент-сервер становится трехзвенной, а в некоторых случаях, она может включать и больше звеньев.



Архитектура клиент-сервер определяет лишь общие принципы взаимодействия между компьютерами, детали взаимодействия определяют различные протоколы. Данная концепция нам говорит, что нужно разделять машины в сети на клиентские, которым всегда что-то надо и на серверные, которые дают то, что надо. При этом взаимодействие всегда начинается клиентом, а правила, по которым происходит взаимодействие описывает протокол.

Существует два вида архитектуры взаимодействия клиент-сервер: первый получил название двухзвенная архитектура клиент-серверного взаимодействия, второй – многоуровневая архитектура клиент-сервер (иногда его называют трехуровневая архитектура или трехзвенная архитектура, но это частный случай).

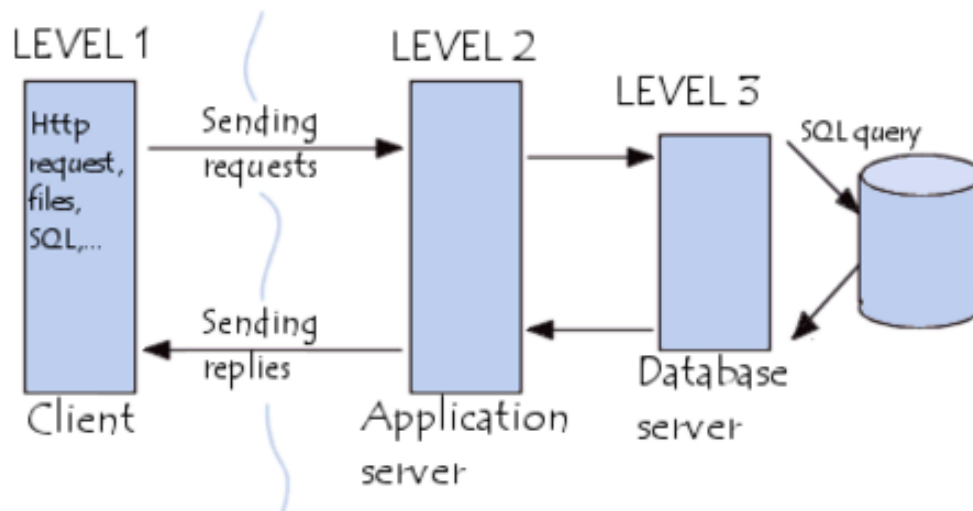
Принцип работы двухуровневой архитектуры взаимодействия клиент-сервер заключается в том, что обработка запроса происходит на одной машине без использования сторонних ресурсов. Двухзвенная архитектура предъявляет жесткие требования к производительности сервера, но в тоже время является очень надежной. Двухуровневую модель взаимодействия клиент-сервер вы можете увидеть на рисунке ниже.



Двухуровневая модель взаимодействия клиент-сервер

Здесь четко видно, что есть клиент (1-ый уровень), который позволяет человеку сделать запрос, и есть сервер, который обрабатывает запрос клиента.

Если говорить про многоуровневую архитектуру взаимодействия клиент-сервер, то в качестве примера можно привести любую современную СУБД (за исключением, наверное, библиотеки SQLite, которая в принципе не использует концепцию клиент-сервер). Суть многоуровневой архитектуры заключается в том, что запрос клиента обрабатывается сразу несколькими серверами. Такой подход позволяет значительно снизить нагрузку на сервер из-за того, что происходит распределение операций, но в то же самое время данный подход не такой надежный, как двухзвенная архитектура. На рисунке ниже вы можете увидеть пример многоуровневой архитектуры клиент-сервер.



Многоуровневая архитектура взаимодействия клиент-сервер

Типичный пример трехуровневой модели клиент-сервер. Если говорить в контексте систем управления базами данных, то первый уровень – это клиент, который позволяет нам писать различные SQL запросы к базе данных. Второй уровень – это движок СУБД, который интерпретирует запросы и реализует взаимодействие между клиентом и файловой системой, а третий уровень – это хранилище данных.

Если мы посмотрим на данную архитектуру с позиции сайта. То первый уровень можно считать браузером, с помощью которого посетитель заходит на сайт, второй уровень – это связка Apache + PHP, а третий уровень – это база данных. Если уж говорить совсем просто, то PHP больше ничего и не делает, кроме как, гоняет строки и базы данных на экран и обратно в базу данных.

Преимущества и недостатки архитектуры клиент-сервер

Преимуществом модели взаимодействия клиент-сервер является то, что программный код клиентского приложения и серверного разделен. Если мы говорим про локальные компьютерные сети, то к преимуществам архитектуры клиент-сервер можно отнести пониженные требования к машинам клиентов, так как большая часть вычислительных операций будет производиться на сервере, а

также архитектура клиент-сервер довольно гибкая и позволяет администратору сделать локальную сеть более защищенной.

К недостаткам модели взаимодействия клиент-сервер можно отнести то, что стоимость серверного оборудования значительно выше клиентского. Сервер должен обслуживать специально обученный и подготовленный человек. Если в локальной сети ложится сервер, то и клиенты не смогут работать (в качестве частного случая можно привести пример: мощности сервера не всегда хватает, чтобы удовлетворит запросы клиентов, если вы хоть раз работали с биллинговыми системами, то понимаете о чем я: время ожидания ответа от сервера может быть очень большим).

Существует несколько популярных СУБД, как платных, так и бесплатных, которые можно рекомендовать для применения в организации. Выполняя поиск, рассмотрите как минимум перечень из десяти СУБД, приведённых ниже, включая отечественные продукты.

1. Oracle 12c

Неудивительно, что корпорация Oracle предлагает одноимённый продукт, с которого обычно начинается рассмотрение вариантов популярных СУБД. Первая версия Oracle была создана в конце 70-х годов. На данный момент у этого продукта блестящая репутация. Кроме того, существует несколько версий этого продукта для удовлетворения потребностей конкретной организации.

Актуальная версия Oracle на момент написания настоящей статьи - 12c - предназначена для облачных сред и может быть размещена на одном или нескольких серверах, это позволяет управлять базами данных, которые содержат миллиарды записей. Некоторые из функций новейшей версии Oracle включают в себя grid framework и использования как физических, так и логических структур.

Это означает, что физическое управление данными не влияет на доступ к логическим структурам. Кроме того, безопасность в этой версии доведена до высочайшего уровня, потому что каждая транзакция изолирована от других.

Достоинства

- Самые свежие инновации и впечатляющий функционал уже внедрены в этом продукте, поскольку компания Oracle стремится держать планку даже на фоне других разработчиков СУБД.
- Оракул является крайне надёжной, фактически это эталон надёжности среди подобных систем.

Недостатки

- Стоимость Oracle может оказаться непомерно высокой, особенно для небольших организаций.
- Система может потребовать значительных ресурсов уже сразу после установки, поэтому возможно потребуется модернизировать оборудование для внедрения Oracle.

Идеально подходит для крупных организаций, которые работают с огромными базами данных и разнообразными функциями.

2. MySQL

MySQL - одна из самых популярных СУБД для веб-приложений. Фактически, является стандартом *de facto* для веб-серверов, которые работают под управлением операционной системы Linux. MySQL - это бесплатный пакет программ, однако новые версии выходят постоянно, расширяя функционал и улучшая безопасность. Существуют специальные платные версии, предназначенные для коммерческого использования. В бесплатной версии наибольший упор делается на скорость и надежность, а не на полноту функционала, который может стать и достоинством и недостатком - в зависимости от области внедрения.

Эта СУБД позволяет выбрать различные движки для системы хранения, которые позволяют менять функционал инструмента и выполнять обработку данных, хранящихся в различных типах таблиц. Гибкость СУБД MySQL обеспечивается поддержкой большого количества типов таблиц: пользователи могут выбрать как таблицы типа MyISAM, поддерживающие полнотекстовый поиск, так и таблицы InnoDB, поддерживающие транзакции на уровне отдельных записей. Более того, СУБД MySQL поставляется со специальным типом таблиц EXAMPLE, демонстрирующим принципы создания новых типов таблиц. Благодаря открытой архитектуре и GPL-лицензированию, в СУБД MySQL постоянно появляются новые типы таблиц. Она также имеет простой в использовании интерфейс, и пакетные команды, которые позволяют удобно обрабатывать огромные объемы данных. Система невероятно надежна и не стремится подчинить себе все доступные аппаратные ресурсы.

Достоинства

- Распространяется бесплатно
- Прекрасно документирована
- Предлагает много функций, даже в бесплатной версии
- Пакет MySQL включен в стандартные репозитории наиболее распространённых дистрибутивов операционной системы Linux, что позволяет устанавливать её элементарно просто
- Поддерживает набор пользовательских интерфейсов
- Может работать с другими базами данных, включая DB2 и Oracle.

Недостатки

- Придётся потратить много времени и усилий, чтобы заставить MySQL выполнять несложные задачи, хотя другие системы делают это автоматически, например: создавать инкрементные резервные копии.
- Отсутствует встроенная поддержка XML или OLAP.
- Для бесплатной версии доступна только платная поддержка.

Идеально подходит для: организаций, которым требуется надежный инструмент управления базами данных, но бесплатный.

3. Microsoft SQL сервер

Ещё одной из популярных СУБД является программный продукт Microsoft SQL-сервер. Это система управления базами данных, движок которой

работает на облачных серверах, а также локальных серверах, причем можно комбинировать типы применяемых серверов одновременно.

Последняя версия Microsoft SQL-сервер поддерживает dynamic data masking (динамическую маскировку данных), которая гарантирует, что только авторизованные пользователи будут видеть конфиденциальные данные.

Достоинства

- Продукт очень прост в использовании
- Текущая версия работает быстро и стабильно
- Движок предоставляет возможность регулировать и отслеживать уровни производительности, которые помогают снизить использование ресурсов.
- Вы сможете получить доступ к визуализации на мобильных устройствах.
- Он очень хорошо взаимодействует с другими продуктами Microsoft.

Недостатки

- Цена для юридических лиц оказывается неприемлемой для большей части организаций
- Даже при тщательной настройке производительности SQL Server способен задействовать все доступные ресурсы
- Сообщается о проблемах с использованием службы интеграции для импорта файлов
- Есть смысл покупать лицензию на этот продукт, если уже внедрена (читай "куплена") экосистема Microsoft.

Идеально подходит для: крупных организаций, которые уже используют ряд продуктов Microsoft.

4. PostgreSQL

PostgreSQL является одним из нескольких бесплатных популярных вариантов СУБД, часто используется для ведения баз данных веб-сайтов. Это весьма старая система, поэтому в настоящее время она хорошо развита, и позволяет пользователям управлять как структурированными, так и неструктурированными данными. Может быть использована на большинстве основных платформ, включая Linux (где особенно хорошо проявляется производительность). Прекрасно справляется с задачами импорта информации из других типов баз данных с помощью собственного инструментария.

Достоинства

- Является масштабируемым решением и позволяет обрабатывать терабайты данных.
- Поддерживает формат json.
- Существует множество predefined функций.
- Доступен ряд интерфейсов.

Недостатки

- Документация туманна, поэтому, возможно, ответы на некоторые вопросы придется искать в интернете.
- Конфигурация может смутить неподготовленного пользователя.

- Скорость работы может падать во время проведения пакетных операций или выполнения запросов чтения.

Идеально подходит для организаций с ограниченным бюджетом, но требует привлечения квалифицированных специалистов, когда требуется возможность выбрать уникальный интерфейс и использовать json.

5. MongoDB

Еще одна бесплатная система, которая имеет коммерческую версию - MongoDB. Считается одним из классических примеров NoSQL-систем, использует JSON-подобные документы и схему базы данных. Написана на языке C++. Она предназначена для приложений, которые используют как структурированные, так и неструктурированные данные. Ядро является очень гибким и работает при подключении базы данных к приложениям через драйверы MongoDB. Существует широкий выбор доступных драйверов, поэтому легко найти драйвер, который будет работать с требуемым языком программирования.

Поскольку изначально система MongoDB не была разработана для обработки моделей реляционных данных, могут возникнуть проблемы производительности, если попытаться использовать её таким образом. Однако, движок предназначен для обработки различных данных, которые нельзя отнести к реляционным, и может хорошо справляться там, где другие движки работают медленно или вообще бессильны.

MongoDB 5.0 - это последняя версия (на июль 2021 г.), и она имеет новую подключаемую систему движков хранения. Документы могут быть проверены в процессе обновления или выполнения вставок, а функции текстового поиска были улучшены. Новая способность частичного индексирования может привести к более высокой производительности, уменьшая размер индексов.

Достоинства

- Скорость и простота в использовании
- Движок поддерживает json и другие традиционные документы NoSQL.
- Данные любой структуры могут быть сохранены/прочитаны быстро и легко.

Недостатки

- SQL не используется в качестве языка запросов.
- Инструменты для перевода SQL-запросов в MongoDB доступны, но их следует рассматривать именно как дополнение.
- Программа установки может занять много времени.

Подходит для организаций, работающих с разнородными данными, которые тяжело поддаются классификации. Для внедрения потребуются высококлассные специалисты.

6. MariaDB

Эта СУБД является бесплатной, но как и многие другие бесплатные приложения, предлагает платные версии. Есть множество доступных плагинов расширений, пожалуй, это самая быстро-развивающаяся СУБД на данный момент.

MariaDB фактически - это ответвление от СУБД MySQL, разрабатываемое сообществом под лицензией GNU GPL.

Ядро базы данных позволяет выбирать из нескольких систем хранения, и это делает использование ресурсов более оптимизированным, что повышает производительность запросов и обработки. В состав MariaDB включена подсистемы хранения данных XtraDB для возможности замены InnoDB, как основной подсистемы хранения. Также включены подсистемы Aria, PBXT и FederateX. Она полностью совместима с MySQL, и вполне подходит в качестве замены, т.к. полностью клонирован как набор команд, так и API. Многие разработчики MySQL были вовлечены в процесс разработки, а сейчас принимают участие в развитии.

Достоинства

- Производительность
- Индикаторы дадут вам знать, как обрабатывается запрос.
- Расширяемая архитектура и плагины позволяют настраивать инструмент в соответствии с вашими потребностями.
- Шифрование доступно в сети, сервере и уровне приложения.

Недостатки

- На данный момент стабильность ниже, чем у MySQL, поэтому даже на новых проектах можно рекомендовать устанавливать mysql.
- Движок довольно новый, поэтому пока нет никаких гарантий дальнейших обновлений.
- Как и во многих других бесплатных базах данных, вам придется платить за поддержку.

Идеальна как альтернатива MySQL, если MySQL не устраивает по каким-то причинам.

7. DB2

Созданная компанией IBM, DB2 представляет собой СУБД, которая имеет возможности NoSQL, и может читать JSON и XML-файлы. Ввиду того, что система разрабатывалась для серверов компании IBM модельного ряда iSeries, логично, что система работает на Windows, Linux и Unix.

Диалект языка SQL, используемый в DB2 за редкими исключениями строго декларативен, система снабжена многофазовым оптимизатором, строящим по этим декларативным конструкциям план выполнения запроса. В диалекте SQL DB2 отсутствуют подсказки оптимизатору, мало развит (а долгое время вообще отсутствовал) язык хранимых процедур, и, таким образом, всё направлено на поддержание декларативного стиля написания запросов. Язык SQL DB2 при этом является вычислительно полным, то есть потенциально позволяет в декларативной форме определять любые вычислимые соответствия между исходными данными и результатом. Это достигается в том числе за счёт использования табличных выражений, рекурсии и других развитых механизмов манипулирования данными.

Оптимизатор DB2 широко использует статистику распределения данных в таблицах (если процесс её сбора был выполнен администратором базы данных), поэтому один и тот же запрос на языке SQL может быть оттранслирован в совершенно различные планы выполнения в зависимости от статистических характеристик данных, которые он обрабатывает.

В рамках концепции повышения уровня интеграции средств безопасности в компьютерной системе, DB2 не имеет собственных средств аутентификации пользователей, интегрируясь со средствами операционной системы или специализированными серверами безопасности. В рамках DB2 осуществляется только авторизация пользователей, аутентифицированных системой.

DB2 является единственной реляционной СУБД общего назначения, имеющей реализации на аппаратно-программном уровне (система IBM i; также в оборудовании мэйнфреймов IBM System z реализуются средства поддержки DB2).

Современные версии DB2 обеспечивают расширенную поддержку использования данных в формате XML, в том числе операции с отдельными элементами документов XML.

Текущая версия DB2 - это LUW 11.1, которая предлагает разнообразные улучшения и доработки. Одно из них, ускорение Blu , которое предназначено, для того чтобы сделать эту базу данных быстрее. Пропуск данных предназначен для повышения быстродействия системы с большим количеством данных, чем может она может вместить в себя. Последняя версия DB2 также обеспечивает усовершенствованные функции аварийного восстановления, совместимости и аналитики.

Достоинства

- Blu Acceleration позволяет грамотно задействовать ресурсы для объёмных баз данных.
- Может быть размещена в облачном хранилище, на физическом сервере, или же и там, и там одновременно.
- Несколько задач могут выполняться одновременно с помощью планировщика задач.
- Коды ошибок и коды завершения позволяют легко отследить, какие задания выполняются или были выполнены с помощью планировщика задач.

Недостатки

- Цена за пределами бюджета многих физических лиц и небольших организаций.
- Сторонние приложения или дополнительное программное обеспечение требуется, для того чтобы заставить функционировать кластеры или несколько вторичных узлов.
- Базовая поддержка доступна только в течение трех лет; после этого, она внезапно становится платной.

Подходит для: крупных организаций, которые планируют выжимать максимум из имеющихся ресурсов и обрабатывают большие БД.

